What are the main components of your harness?

For the harness, it will try to start up to five tasks concurrently. The general structure is that I first let the expert explore the problem and implement the infrastructure. When the expert conducts 25 runs or hits the budget threshold that I preset, I will switch to the standard model to perform the longer verification-and-repair loop. When the standard is finished, the harness will let the expert review its implementations if we have budget. When there is a stall, if we have budget, the harness will also involve the expert model to seek advice. In terms of the tools that I use, they are for reading files, running commands, replacing text, creating files, and finishing.

Other than coordinating the different models based on budget or a preset sequence and doing tool calling, the harness also took charge of deciding when to compact the context as well as limiting large outputs, so that the context will not be flooded with unnecessary information. It also did some preliminary syntax checks after the edits to catch some syntax mistakes early. It also get to record repetitive failures.

What optimizations and trade-offs did you make?

There are several trade-offs that I made. The first is the trade-off between cost and accuracy. Previously, I only used the standard model. It was cheap, but was incapable solving a lot of tasks. When I switched to the current version, there was a large jump in the accuracy. Thus, for the early part of building the foundation as well as the plan, the expert model is more favored. Another trade-off that I make is how early should I compact the context to compact early or not, since compacting early saves tokens. I tried to compact early at 30k in a revision comparing to 40k in current version. The compact-early version did not do well. I think that it might because summarization costs calls, may omit details, and can lead to rereading files.

For the optimizations, I did several. Firstly, we tried to compact the standard conversations at around 40k tokens to control accumulated history. We allow it to carry forward notes, requirements, current changes and selected evidence. Secondly, we have early error detection, such as validating edit arguments and running syntax checks. This allows us to try to repair the problem when there is one. Last but not least, we try to optimize the involvement of the expert and standard models. We tried several ratios of expert to standard and considered the submitted version as the best, though it might be based on randomness.

What pitfalls did you run into, and what helped?

The helping part is that I let the expert implement first and then let the standard model read the previous conversation and continue the work. This part boosts the accuracy and time per solved task. I also chatted with my friends. They also claimed that this part helped.

The first pitfall that I ran into is that I wanted the expert model to provide a plan for the later standard to follow. However, the expert sometimes provides an unusable plan. The second pitfall is that I treated a promising local result as evidence of a reliable leaderboard improvement. Some versions have good local results (tasks in SWE-bench pro). However, when I switched to the leaderboard, it behaved differently. Last but not least, the other pitfall is that spending on the expert model without appropriate structure also cannot solve more tasks. I tried the expert model in one version, but the money was spent fast without great improvement.